

IT3025: Introduction to Agentic AI

Study notes — course overview, Modules 1-3, Lectures 4-5

Course Overview

IT3025: Introduction to Agentic AI is a course in the School of Information Technology at the University of Cincinnati. It meets twice a week for 80-minute sessions and is built around hands-on Jupyter notebooks rather than pure lecture - most weeks pair a short lecture with a lab you build yourself, step by step, with no agent framework hiding the details.

Module 1: What Is Agentic AI?

A system is agentic when it combines multi-step reasoning, autonomy, real actions, and verification, built on an "augmented LLM" that can use retrieval, tools, and memory. An AI Agent is one autonomous decision-maker with instructions, guardrails, and tools; Agentic AI is the broader system design built from decisions like that, sometimes chaining many agents together.

The quick test for workflow versus agent: if you can draw the steps on a whiteboard before running it, it's a workflow; if the path depends on what happens mid-task, it's an agent. Module 1 also gave a first hands-on taste of agent-like behavior using n8n, a visual, node-based workflow tool, before writing any framework code.

Module 2: Python Foundations

Module 2 covered the Python foundations needed to build agents, aimed at students coming from Java: indentation (not braces) defines blocks, variables are dynamically typed, and the key collections are list, dict, tuple, and set - dict especially, since agent messages and tool calls are JSON under the hood.

Functions support default parameter values and `*args/**kwargs` for flexible agent and tool wrappers. The module also set up the toolchain used all course: Jupyter notebooks (.ipynb) for interactive, cell-by-cell prototyping, Cursor as the AI-native code editor, and uv as the fast, modern replacement for pip and venv.

Module 3: Agentic Design Patterns

Module 3 covered five essential agentic design patterns: Prompt Chaining (an ordered sequence of dependent LLM calls, where each step's prompt is built from the previous step's output), Routing (classify the input, then send it down a specialized path), Parallelization (run independent calls at once and combine the results), Orchestrator-Worker (a lead call dynamically assigns work to worker calls), and Evaluator-Optimizer (draft, critique against explicit criteria, then revise, with a bounded retry limit).

The core insight is that reliability comes from autonomy and tool integration working together - more autonomy is not automatically better. This module also introduced Azure OpenAI via the course's

APIM gateway (using a deployment name instead of a plain model id) and Lab 2, which builds a small Prompt Chaining program by hand.

Lecture 4: LLM Providers

An LLM is a neural network trained to predict text one token at a time; a token is the basic unit it reads, writes, and is billed by, and every model has a context window - a hard limit on how many tokens of prompt plus reply it can handle at once. Cost is driven separately by input tokens and (usually pricier) output tokens, and cost efficiency means quality per dollar, not just the lowest sticker price.

The lecture walked through OpenAI, Anthropic, and Gemini as flagship paid APIs, OpenRouter as one API key that reaches many providers (including free models), and Ollama for running small open-source models locally with no API key at all. Lab 3 compares the same question across several of these providers.

Lecture 5: Tools and the Agent Loop

An agent = an LLM + a harness: the LLM only ever generates text, and the harness - ordinary code - is what calls the model, holds the conversation, and runs whatever the model asks for. A tool is just a plain Python function paired with a JSON description so the model can ask for it by name; the model never runs the code itself, only describes wanting to call it.

The agent loop is the harness checking, in a while loop instead of a single if, whether the model wants a tool, running it, and calling the model again - repeating until it gives a final answer. Lab 4 puts this all together by hand-building Study Buddy, a persona-based quiz agent with a system prompt, a Gradio chat UI, a tool, and a full agent loop.